

Fix Movement Boundary Bugs

YOU

This is the project assigned to me what im expected to do is go through all this , then fix the intended bugs and fix it , we'll do it one by one

CHATGPT

Yep. I've gone through the assignment brief and the starter code. We can do this **one task at a time**, and I'll explain what the bug is, why it happens, what we should change, and then we'll implement and test it.

The assignment explicitly says there is **1 deliberate bug with two symptoms**, followed by **3 features** to implement: movement/boundaries, collision/game-over, distance scoring, and the shield mechanic. `filecite{turn0file1}{L52-L83}`

Current project structure

- `main.py` → main Pygame loop. `filecite{turn0file0}{L15-L36}`
- `game_engine.py` → coordinates helicopter, obstacles, spawning, updating, drawing. `filecite{turn0file2}{L23-L55}`
- `helicopter.py` → helicopter movement and boundaries. `filecite{turn0file3}{L11-L35}`
- `obstacle.py` → obstacle movement and collision rectangles. `filecite{turn0file4}{L9-L31}`
- `renderer.py` → all visual rendering. `filecite{turn0file5}{L16-L30}`

And importantly, the starter already tells us exactly what the intended requirements are.

`filecite{turn0file1}{L109-L122}`

We'll tackle it like this

Task 1 → Movement + boundary bug

1. Understand the existing `vy` behavior.
2. Fix unlimited acceleration.
3. Make switching Up ↔ Down responsive.
4. Add the missing bottom boundary.
5. Test it.

Task 2 → Collision + Game Over

1. Detect helicopter/obstacle intersection.
2. Make the gap safe.
3. Freeze/end the game on collision.
4. Display game-over.

Task 3 → Distance score

1. Decide what "distance traveled" means in this game.

2. Track it while playing.
3. Render it.
4. Show final distance.
5. Reset on restart.

Task 4 → Shield

1. Add activation input.
2. Show shield visually.
3. Consume it on exactly one collision.
4. Make the helicopter vulnerable again afterward.

One important thing: **we shouldn't blindly modify everything at once.** The assignment specifically wants an iterative LLM debugging process and critical code review.

So we'll preserve that process.

Let's start with Task 1

The current movement code is:

```
if keys_pressed[pygame.K_UP]:
    self.vy -= THRUST
if keys_pressed[pygame.K_DOWN]:
    self.vy += THRUST
```

with:

```
THRUST = 0.4
```

So if you hold Up for a while:

```
vy = 0
-0.4
-0.8
-1.2
-1.6
-2.0
...
```

There is **no maximum velocity**. That's exactly why holding Up and then pressing Down doesn't immediately make the helicopter descend. Down has to first cancel all that negative velocity. The README describes this exact issue. `filecite[turn0file1L59-L72]`

There's also only a **top boundary**:

```
if self.y < 0:
    self.y = 0
    self.vy = 0
```

and nothing for the bottom. `filecite[turn0file3L25-L30]`

So Task 1 is actually two fixes:

Unlimited acceleration

↓

cap vy

Missing bottom boundary

↓

clamp helicopter to bottom

I'd suggest we **first implement Task 1 only**, run the game, and verify the behavior before touching collisions.

If you're ready, I'll give you the exact `helicopter.py` change and explain why each line is there.

YOU

let's do it

CHATGPT

Perfect. Let's fix **Task 1 only** first.

1. What we're changing

The assignment wants two things:

- Cap the helicopter's vertical speed so `vy` cannot grow indefinitely.
- Add a bottom boundary so the helicopter cannot leave the screen.

Right now `handle_input()` keeps adding ± 0.4 every frame, while `update()` only handles the top edge.

2. Modify `helicopter.py`

I'd use a maximum vertical speed of `5.0`. This gives us:

```
THRUST = 0.4
MAX_VY = 5.0
```

Then change `handle_input()` to:

```
def handle_input(self, keys_pressed):
    if keys_pressed[pygame.K_UP]:
        self.vy -= THRUST
    if keys_pressed[pygame.K_DOWN]:
        self.vy += THRUST

    self.vy = max(-MAX_VY, min(MAX_VY, self.vy))
```

And change `update()` to:

```
def update(self, height_bound):
    self.y += self.vy

    if self.y < 0:
        self.y = 0
        self.vy = 0
```

```
if self.y > height_bound:  
    self.y = height_bound  
    self.vy = 0
```

So the relevant part of the file becomes:

```
THRUST = 0.4
```

```
MAX_VY = 5.0
```

```
class Helicopter:
```

```
    def __init__(self, x, y, width=40, height=24):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.width = width
```

```
        self.height = height
```

```
        self.vy = 0.0
```

```
    def handle_input(self, keys_pressed):
```

```
        if keys_pressed[pygame.K_UP]:
```

```
            self.vy -= THRUST
```

```
        if keys_pressed[pygame.K_DOWN]:
```

```
            self.vy += THRUST
```

```
        self.vy = max(-MAX_VY, min(MAX_VY, self.vy))
```

```
    def update(self, height_bound):
```

```
        self.y += self.vy
```

```
        if self.y < 0:
```

```
            self.y = 0
```

```
            self.vy = 0
```

```
        if self.y > height_bound:
```

```
            self.y = height_bound
```

```
            self.vy = 0
```

Why this works

The important line is:

```
self.vy = max(-MAX_VY, min(MAX_VY, self.vy))
```

It clamps velocity to:

$$-5 \leq v_y \leq +5$$

So holding Up for 10 seconds doesn't produce something ridiculous like $v_y = -120$.

And when you switch from Up to Down, the maximum amount of old momentum that needs to be cancelled is only 5, rather than an arbitrarily large value.

For the boundary:

```
if self.y > height_bound:
```

catches the bottom just like the existing code catches the top.

One small thing to notice

We're **not changing** `game_engine.py` **or** `main.py` for Task 1. `GameEngine` already passes the screen height into:

```
self.helicopter.update(HEIGHT)
```

so the information needed for the boundary check is already being supplied.

filecite0turn0file20L43-L4401

Now test it

Run:

```
python main.py
```

Then test these **three cases**:

1. Hold **Up** for several seconds → helicopter should stop accelerating upward.
2. Hold **Down** for several seconds → helicopter should stop at the bottom.
3. Hold **Up**, then immediately hold **Down** → it should reverse much more quickly instead of continuing to drift upward.

Don't move on to Task 2 yet. **Run these tests first**, and tell me what happens. If something feels

wrong, we'll debug it before touching collision detection.

YOU

this is done , now on to the next one

CHATGPT

Nice. Task 1 is out of the way. ✅

Task 2: Obstacle collision + Game Over

This is the next feature the README asks us to implement:

Touching either the top or bottom wall of an obstacle should end the game, while flying through the gap should not.

What we already have

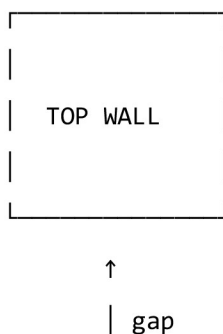
The good news is that **most of the collision infrastructure already exists**.

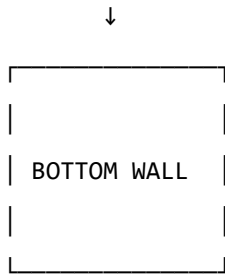
Each obstacle has two rectangles:

```
def get_top_rect(self):
    top_height = self.gap_y - self.gap_height / 2
    return pygame.Rect(int(self.x), 0, self.wall_width, int(top_height))

def get_bottom_rect(self):
    bottom_y = self.gap_y + self.gap_height / 2
    return pygame.Rect(
        int(self.x),
        int(bottom_y),
        self.wall_width,
        int(self.screen_height - bottom_y)
    )
```

So an obstacle is effectively:





That's already implemented. [filecite02turn0file402L25-L31](#)

And the helicopter already has a `get_rect()` method that gives us its collision rectangle.

[filecite02turn0file302L32-L35](#)

So we **don't need to reinvent either of those**.

What is missing?

Look at `GameEngine.update()` :

```
def update(self):
    self.helicopter.update(HEIGHT)

    self.frames_until_spawn -= 1
    if self.frames_until_spawn <= 0:
        self._spawn_obstacle()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for obstacle in self.obstacles:
        obstacle.update()

    self.obstacles = [o for o in self.obstacles if not o.is_off_screen()]
```

It updates everything, but there's **no collision check**. That's why the helicopter can currently fly straight through obstacles. [filecite02turn0file202L43-L53](#)

We need to add:

```
helicopter.rectangle
↓
```

```

check against top wall
    ↓
check against bottom wall
    ↓
collision?
  ✓      ↘
YES      NO
  ↓      ↓
game over  continue

```

Step 1: Add a game-over state

Inside `GameEngine.__init__()` add:

```
self.game_over = False
```

So:

```

def __init__(self):
    self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
    self.obstacles = []
    self.frames_until_spawn = 0
    self.game_over = False

```

Step 2: Add collision detection

Inside `update()` , after the obstacles have been updated, we can check them:

```

heli_rect = self.helicopter.get_rect()

for obstacle in self.obstacles:
    obstacle.update()

    if (
        heli_rect.colliderect(obstacle.get_top_rect())
        or heli_rect.colliderect(obstacle.get_bottom_rect())

```

```
):  
    self.game_over = True
```

But there's one important issue.

Once `game_over` becomes `True`, we don't want the game continuing to update normally.

So I'd structure `update()` like this:

```
def update(self):  
    if self.game_over:  
        return  
  
    self.helicopter.update(HEIGHT)  
  
    self.frames_until_spawn -= 1  
    if self.frames_until_spawn <= 0:  
        self._spawn_obstacle()  
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES  
  
    for obstacle in self.obstacles:  
        obstacle.update()  
  
    heli_rect = self.helicopter.get_rect()  
  
    for obstacle in self.obstacles:  
        if (  
            heli_rect.colliderect(obstacle.get_top_rect())  
            or heli_rect.colliderect(obstacle.get_bottom_rect())  
        ):  
            self.game_over = True  
            break  
  
    self.obstacles = [  
        o for o in self.obstacles  
        if not o.is_off_screen()  
    ]
```

Why `colliderect()` ?

Pygame gives us:

```
rect1.colliderect(rect2)
```

which returns `True` when the two rectangles overlap.

So:

```
heli_rect.colliderect(obstacle.get_top_rect())
```

means:

```
"Is the helicopter touching the top wall?"
```

And:

```
heli_rect.colliderect(obstacle.get_bottom_rect())
```

means:

```
"Is the helicopter touching the bottom wall?"
```

If both are false, the helicopter is inside the gap and continues safely.

Step 3: Display Game Over

We already have a renderer function specifically designed for banners:

```
def draw_banner(surface, font, text):  
    surf = font.render(text, True, (180, 40, 40))  
    rect = surf.get_rect(  
        center=(surface.get_width() // 2, surface.get_height() // 2)  
    )  
    surface.blit(surf, rect)
```

So we should **reuse it**, rather than creating another rendering system. [filecite00](#)turn0file5L24-

L30 

Change `GameEngine.draw()` from:

```
def draw(self, surface, font):  
    from game import renderer  
    renderer.draw_scene(surface, self.helicopter, self.obstacles)
```

to:

```
def draw(self, surface, font):  
    from game import renderer  
  
    renderer.draw_scene(  
        surface,  
        self.helicopter,  
        self.obstacles  
    )  
  
    if self.game_over:  
        renderer.draw_banner(  
            surface,  
            font,  
            "GAME OVER"  
        )
```

So your `game_engine.py` should now look like this

```
import random  
  
from game.helicopter import Helicopter  
from game.obstacle import Obstacle  
from game.renderer import WIDTH, HEIGHT  
  
SPAWN_INTERVAL_FRAMES = 90
```

```
GAP_HEIGHT = 150
```

```
WALL_WIDTH = 60
```

```
SCROLL_SPEED = 3
```

```
class GameEngine:
```

```
    def __init__(self):
```

```
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
```

```
        self.obstacles = []
```

```
        self.frames_until_spawn = 0
```

```
        self.game_over = False
```

```
    def _spawn_obstacle(self):
```

```
        margin = 60
```

```
        gap_y = random.randint(
```

```
            margin + GAP_HEIGHT // 2,
```

```
            HEIGHT - margin - GAP_HEIGHT // 2
```

```
        )
```

```
        self.obstacles.append(
```

```
            Obstacle(
```

```
                x=WIDTH,
```

```
                gap_y=gap_y,
```

```
                gap_height=GAP_HEIGHT,
```

```
                wall_width=WALL_WIDTH,
```

```
                screen_height=HEIGHT,
```

```
                speed=SCROLL_SPEED
```

```
            )
```

```
        )
```

```
    def handle_input(self, keys_pressed):
```

```
        self.helicopter.handle_input(keys_pressed)
```

```
    def handle_keydown(self, key):
```

```
        pass
```

```
    def update(self):
```

```
    if self.game_over:
        return

    self.helicopter.update(HEIGHT)

    self.frames_until_spawn -= 1

    if self.frames_until_spawn <= 0:
        self._spawn_obstacle()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for obstacle in self.obstacles:
        obstacle.update()

    heli_rect = self.helicopter.get_rect()

    for obstacle in self.obstacles:
        if (
            heli_rect.colliderect(obstacle.get_top_rect())
            or heli_rect.colliderect(obstacle.get_bottom_rect())
        ):
            self.game_over = True
            break

    self.obstacles = [
        o for o in self.obstacles
        if not o.is_off_screen()
    ]

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.helicopter,
        self.obstacles
    )
```

```
if self.game_over:
    renderer.draw_banner(
        surface,
        font,
        "GAME OVER"
    )
```

Test these cases

Don't move on to Task 3 until these work:

Test 1: Fly through the middle of an obstacle's gap.

→ Game continues.

Test 2: Hit the top wall.

→ GAME OVER appears.

Test 3: Hit the bottom wall.

→ GAME OVER appears.

Test 4: After Game Over, keep holding Up/Down.

→ The game should remain frozen because `update()` immediately returns once `game_over` is true.

One thing you'll notice after this task: **there is currently no restart mechanism.** That's okay for now. The assignment's scoring requirement says starting a *new game* must reset the score, so we'll deal with restart when we build the game state/score system rather than prematurely adding it here.

YOU

done , now on to the next one

CHATGPT

Yep. Task 2 is done. Let's move to **Task 3: Distance-based scoring.**

The assignment specifically wants the score to represent how far the helicopter has traveled, increase automatically during gameplay, show the final distance on game over, and reset when a new game starts.

First: what should "distance" mean here?

The helicopter's `x` position doesn't change, so we **cannot** calculate distance from the helicopter's position.

Instead, the world is moving toward the helicopter. Obstacles move left at:

```
SCROLL_SPEED = 3
```

So every frame, the game effectively advances by **3 pixels of world distance**.

That gives us a simple score:

```
distance += SCROLL_SPEED
```

every frame while the game is running.

We can display it as:

```
Distance: 1234
```

Step 1: Add distance to `GameEngine`

In `__init__()`:

```
self.distance = 0
```

So:

```
def __init__(self):
    self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
    self.obstacles = []
```

```
self.frames_until_spawn = 0
self.game_over = False
self.distance = 0
```

This automatically gives every newly-created `GameEngine` a score of zero.

Step 2: Increase it during gameplay

In `update()` , after checking `game_over` , increase the distance:

```
self.distance += SCROLL_SPEED
```

So:

```
def update(self):
    if self.game_over:
        return

    self.distance += SCROLL_SPEED

    self.helicopter.update(HEIGHT)

    ...
```

This is important because we put it **after** the `game_over` check.

Therefore:

Playing

↓

distance increases

Game Over

↓

update() returns

↓

distance stops

That's exactly what we want.

Step 3: Display the current distance

We already have `renderer.draw_text()` :

```
def draw_text(surface, font, text, pos, color=COLOR_TEXT):  
    surface.blit(font.render(text, True, color), pos)
```

so we can reuse that. [filecite02turn0file502L24-L2601](#)

Modify `GameEngine.draw()` :

```
def draw(self, surface, font):  
    from game import renderer  
  
    renderer.draw_scene(  
        surface,  
        self.helicopter,  
        self.obstacles  
    )  
  
    renderer.draw_text(  
        surface,  
        font,  
        f"Distance: {self.distance}",  
        (10, 10)
```

```
)

if self.game_over:
    renderer.draw_banner(
        surface,
        font,
        f"GAME OVER - Distance: {self.distance}"
    )
```

Now while playing you'll see:

```
Distance: 3
```

then:

```
Distance: 6
```

then:

```
Distance: 9
```

and so on.

When you crash:

```
GAME OVER - Distance: 1234
```

One improvement I'd make

The assignment says the score should be **distance**, not necessarily raw pixels.

`filecite0turn0file102L94-L9501`

Raw pixels are technically fine, but something like:

```
Distance: 12.3 m
```

is more readable.

However, the starter project doesn't define a pixels-to-metres conversion, so **we should not invent one**.

Therefore, for this assignment, keeping it as a simple distance unit is safest:

```
Distance: 1234
```

Step 4: What about resetting?

The nice part is that our architecture already makes this easy.

A new game is represented by creating a new `GameEngine` :

```
engine = GameEngine()
```

and `__init__()` sets:

```
self.distance = 0
```

So the score naturally resets to zero.

But: our current `main.py` doesn't actually provide a restart key yet. The assignment requires the score to reset when starting a new game, so we'll need a restart mechanism.

I'd **not add that separately right now**. The shield task also needs an activation key and game-state handling, so we'll add the restart behavior cleanly when we finish the remaining game-state work.

Your Task 3 changes

In short:

```
# __init__
self.distance = 0

# update
if self.game_over:
    return
```

```
self.distance += SCROLL_SPEED
```

and in `draw()` :

```
renderer.draw_text(
    surface,
    font,
    f"Distance: {self.distance}",
    (10, 10)
)

if self.game_over:
    renderer.draw_banner(
        surface,
        font,
        f"GAME OVER - Distance: {self.distance}"
    )
```

Test that:

1. Distance starts at 0 .
2. It increases while flying.
3. It stops when you crash.
4. The final value is visible on the game-over screen.

Once that's working, **Task 4 is the shield mechanic**, which is the last actual feature in the README. [filecite0turn0file1L99-L105](#)

YOU

so we have the shielf thing remaining and also the restart button right , let's do it

CHATGPT

Exactly. We have **two things left**:

1. 🛡️ **Shield mechanic**: activate it, visibly show it, absorb exactly one obstacle collision, then disappear.
2. 🔄 **Restart**: restart after game over and reset the game state, including distance.

The README explicitly requires the shield to protect against **one collision only** and turn off immediately after absorbing the hit. [filecite turn0file1L99-L105](#)

Let's implement the **shield first**, then add restart cleanly around the same game-state logic.

Task 4A: Shield

1. Add shield state

In `GameEngine.__init__()` add:

```
self.shield_active = False
```

So you'll have:

```
def __init__(self):
    self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
    self.obstacles = []
    self.frames_until_spawn = 0
    self.game_over = False
    self.distance = 0
    self.shield_active = False
```

2. Give the shield a key

We'll use **Space** because it's intuitive and doesn't interfere with Up/Down.

Your `handle_keydown()` is currently:

```
def handle_keydown(self, key):
    pass
```

Change it to:

```
def handle_keydown(self, key):  
    if key == pygame.K_SPACE and not self.game_over:  
        self.shield_active = True
```

Since we're using `pygame.K_SPACE`, make sure `game_engine.py` has:

```
import pygame
```

at the top.

So the imports become:

```
import random  
import pygame  
  
from game.helicopter import Helicopter  
from game.obstacle import Obstacle  
from game.renderer import WIDTH, HEIGHT
```

What this means

Press Space:

```
SPACE  
↓  
shield_active = True
```

The shield stays active until it actually absorbs a collision.

3. Modify collision handling

Currently we have something conceptually like:

```
if collision:  
    self.game_over = True
```

We need to change that logic to:

```
Collision?
|
|— Shield active?
|   |
|   |— YES → consume shield → continue playing
|   |
|   |— NO → game over
```

Replace the collision section with:

```
heli_rect = self.helicopter.get_rect()

for obstacle in self.obstacles:
    if (
        heli_rect.colliderect(obstacle.get_top_rect())
        or heli_rect.colliderect(obstacle.get_bottom_rect())
    ):
        if self.shield_active:
            self.shield_active = False
        else:
            self.game_over = True

        break
```

The `break` is important because we don't want to process more obstacles after the collision has already been handled.

Result

If shield is active:

```
* collision
↓
shield = False
↓
```

```
game continues
```

If shield isn't active:

```
* collision
↓
game_over = True
```

That's exactly the "one collision" behavior required by the assignment.

4. Show the shield

We need a visual indication while it's active.

The cleanest place is `renderer.py`, because the starter code deliberately keeps drawing there.

```
def draw_scene(surface, helicopter, obstacles):
```

Change:

```
def draw_scene(surface, helicopter, obstacles):
```

to:

```
def draw_scene(surface, helicopter, obstacles, shield_active=False):
```

Then after drawing the helicopter:

```
pygame.draw.rect(
    surface,
    COLOR_HELI,
    helicopter.get_rect(),
    border_radius=4
)

if shield_active:
    rect = helicopter.get_rect()
    pygame.draw.circle(
```

```
        surface,  
        (30, 120, 255),  
        rect.center,  
        max(rect.width, rect.height)  
    )
```

However, **there's a small visual issue** with that order: the circle would be drawn *after* the helicopter and could cover it.

So better:

```
def draw_scene(surface, helicopter, obstacles, shield_active=False):  
    surface.fill(COLOR_BG)  
  
    for obstacle in obstacles:  
        pygame.draw.rect(  
            surface,  
            COLOR_OBSTACLE,  
            obstacle.get_top_rect()  
        )  
        pygame.draw.rect(  
            surface,  
            COLOR_OBSTACLE,  
            obstacle.get_bottom_rect()  
        )  
  
    if shield_active:  
        rect = helicopter.get_rect()  
        pygame.draw.circle(  
            surface,  
            (30, 120, 255),  
            rect.center,  
            max(rect.width, rect.height)  
        )  
  
    pygame.draw.rect(  
        surface,
```

```
        COLOR_HELI,  
        helicopter.get_rect(),  
        border_radius=4  
    )
```

Now the shield appears **behind** the helicopter.

5. Tell renderer whether the shield is active

In `GameEngine.draw()` change:

```
renderer.draw_scene(  
    surface,  
    self.helicopter,  
    self.obstacles  
)
```

to:

```
renderer.draw_scene(  
    surface,  
    self.helicopter,  
    self.obstacles,  
    self.shield_active  
)
```

You can also display text:

```
if self.shield_active:  
    renderer.draw_text(  
        surface,  
        font,  
        "SHIELD ACTIVE",  
        (10, 40)  
    )
```

That makes the state very obvious when recording your final video.

Now the restart button

This is where we can make the game much cleaner.

We'll use **R** to restart after game over.

The important idea is:

Instead of manually resetting 10 different variables, create a fresh `GameEngine` .

That's much less error-prone.

6. Add restart handling

Change `handle_keydown()` to:

```
def handle_keydown(self, key):
    if key == pygame.K_r and self.game_over:
        self.__init__()
        return

    if key == pygame.K_SPACE and not self.game_over:
        self.shield_active = True
```

So:

```
R + Game Over
    ↓
new GameEngine state
    ↓
distance = 0
shield = False
obstacles = []
game_over = False
helicopter.reset
```

This automatically resets everything because `__init__()` initializes all of those values.

7. Make the restart instruction visible

Change the game-over banner from:

```
renderer.draw_banner(  
    surface,  
    font,  
    f"GAME OVER - Distance: {self.distance}"  
)
```

to:

```
renderer.draw_banner(  
    surface,  
    font,  
    f"GAME OVER - Distance: {self.distance} | Press R to Restart"  
)
```

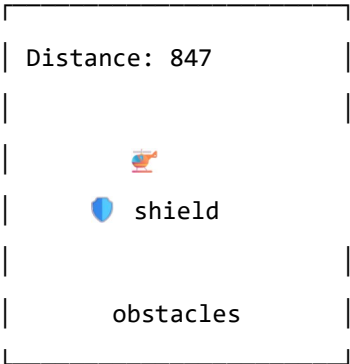
One issue: that string is fairly long for a 700px window.

I'd actually make the banner two lines using the existing `draw_text()` :

```
if self.game_over:  
    renderer.draw_banner(  
        surface,  
        font,  
        f"GAME OVER - Distance: {self.distance}"  
    )  
  
    renderer.draw_text(  
        surface,  
        font,  
        "Press R to Restart",  
        (surface.get_width() // 2 - 100, surface.get_height() // 2 + 35)  
    )
```

Final behavior

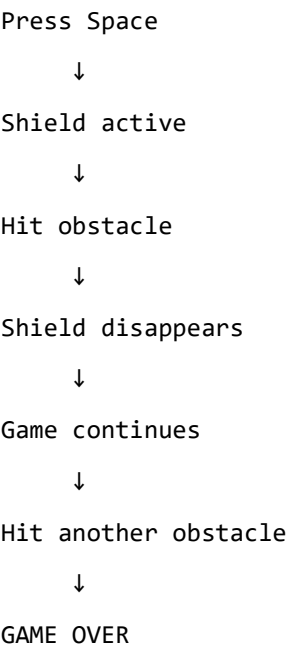
You'll now have:



Controls:

Key	Action
↑	Move up
↓	Move down
Space	Activate shield
R	Restart after game over

And the shield behavior is:



That matches the assignment's requirement that the shield absorbs **exactly one collision** and the helicopter becomes vulnerable again afterward. [filecite\turn0file1\L101-L105](#)

One thing to test carefully

Don't just test "shield works." Test this exact sequence:

1. Start game.
2. Press **Space**.
3. Confirm shield is visibly active.
4. Deliberately hit an obstacle.
5. Confirm **no game over**.
6. Confirm shield disappears.
7. Hit another obstacle.
8. Confirm **GAME OVER**.
9. Press **R**.
10. Confirm helicopter starts fresh and **Distance is 0**.

If all of those pass, we've essentially completed the actual programming requirements. The README's expected behavior covers these cases too. [filecite\turn0file1\L109-L122](#)

YOU

for 3.

CHATGPT

If you mean **step 3 of the shield implementation, modifying the collision handling**, here's exactly what to change.

In `game_engine.py`, find your current collision section inside `update()` :

```
heli_rect = self.helicopter.get_rect()

for obstacle in self.obstacles:
    if (
        heli_rect.colliderect(obstacle.get_top_rect())
        or heli_rect.colliderect(obstacle.get_bottom_rect())
    ):
        self.game_over = True
        break
```

Replace it with:

```
heli_rect = self.helicopter.get_rect()

for obstacle in self.obstacles:
    if (
        heli_rect.colliderect(obstacle.get_top_rect())
        or heli_rect.coll
```

YOU

for 3.

what do i change here

```
def update(self):
    if self.game_over:
        return
    self.distance += SCROLL_SPEED

    self.helicopter.update(HEIGHT)

    self.frames_until_spawn -= 1

    if self.frames_until_spawn <= 0:
        self._spawn_obstacle()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for obstacle in self.obstacles:
        obstacle.update()

    heli_rect = self.helicopter.get_rect()

    for obstacle in self.obstacles:
        if (
            heli_rect.colliderect(obstacle.get_top_rect())
            or heli_rect.colliderect(obstacle.get_bottom_rect())
        ):
            self.game_over = True
            break

    self.obstacles = [
        o for o in self.obstacles
        if not o.is_off_screen()
    ]
```

CHATGPT

Yep, you're at exactly the right place. For **step 3 of the shield**, you only need to replace the collision block.

Replace this:

```
heli_rect = self.helicopter.get_rect()

for obstacle in self.obstacles:
    if (
        heli_rect.colliderect(obstacle.get_top_rect())
        or heli_rect.colliderect(obstacle.get_bottom_rect())
    ):
        self.game_over = True
        break
```

With this:

```
heli_rect = self.helicopter.get_rect()

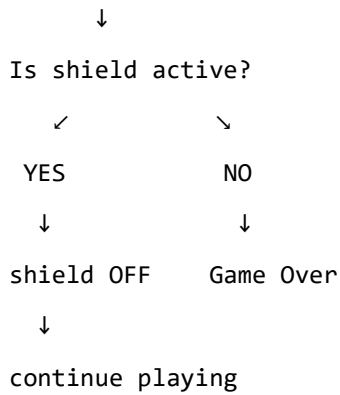
for obstacle in self.obstacles:
    if (
        heli_rect.colliderect(obstacle.get_top_rect())
        or heli_rect.colliderect(obstacle.get_bottom_rect())
    ):
        if self.shield_active:
            self.shield_active = False
        else:
            self.game_over = True

        break
```

That's it for **step 3**.

The logic is now:

Collision happens



Your full `update()` should therefore currently look like:

```

def update(self):
    if self.game_over:
        return

    self.distance += SCROLL_SPEED

    self.helicopter.update(HEIGHT)

    self.frames_until_spawn -= 1

    if self.frames_until_spawn <= 0:
        self._spawn_obstacle()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for obstacle in self.obstacles:
        obstacle.update()

    heli_rect = self.helicopter.get_rect()

    for obstacle in self.obstacles:
        if (
            heli_rect.colliderect(obstacle.get_top_rect())
            or heli_rect.colliderect(obstacle.get_bottom_rect())
        ):
            if self.shield_active:
                self.shield_active = False

```

```
        else:
            self.game_over = True

        break

self.obstacles = [
    o for o in self.obstacles
    if not o.is_off_screen()
]
```

Don't change anything else in `update()` yet.

Next we'll do **step 4: making the shield visually appear**, in `renderer.py`.

YOU

after the 3rd one , tell me exactly where each change is supposed to be there and what change

CHATGPT

Absolutely. From here on, I'll be **very explicit about the file, exact location, what to find, what to replace/add, and what the final section should look like**. No more vague "change this somewhere." 👍

For the shield, we've completed **Step 3: collision handling** in `game_engine.py`.

Step 4: Make the shield visible

File to edit

`game/renderer.py`

Find this function

You currently have:

```
def draw_scene(surface, helicopter, obstacles):
    surface.fill(COLOR_BG)
    for obstacle in obstacles:
        pygame.draw.rect(surface, COLOR_OBSACLE, obstacle.get_top_rect())
        pygame.draw.rect(surface, COLOR_OBSACLE, obstacle.get_bottom_rect())
    pygame.draw.rect(surface, COLOR_HELI, helicopter.get_rect(), border_radius=4)
```

This is the function responsible for drawing the game scene. `fileciteurn0file5L16-L21`

Change 1: Add a parameter

Change:

```
def draw_scene(surface, helicopter, obstacles):
```

to:

```
def draw_scene(surface, helicopter, obstacles, shield_active=False):
```

The `shield_active=False` means the function still works normally if we don't provide a shield value.

Change 2: Add the shield drawing

Now replace the **entire** `draw_scene()` **function** with:

```
def draw_scene(surface, helicopter, obstacles, shield_active=False):
    surface.fill(COLOR_BG)

    for obstacle in obstacles:
        pygame.draw.rect(surface, COLOR_OBSACLE, obstacle.get_top_rect())
        pygame.draw.rect(surface, COLOR_OBSACLE, obstacle.get_bottom_rect())

    if shield_active:
        rect = helicopter.get_rect()
        pygame.draw.circle(
            surface,
            (30, 120, 255),
            rect.center,
            max(rect.width, rect.height)
        )

    pygame.draw.rect(
        surface,
        COLOR_HELI,
        helicopter.get_rect(),
        border_radius=4
    )
```

Why we're putting the shield before the helicopter

The drawing order is:

```
obstacles
  ↓
shield
  ↓
helicopter
```

So the shield appears **behind** the helicopter rather than covering it.

Step 5: Tell the renderer whether the shield is active

File to edit

game/game_engine.py

Find this function

You currently have something like:

```
def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.helicopter,
        self.obstacles
    )

    ...
```

Your `draw_scene()` call currently doesn't know whether the shield is active.

Change this:

```
renderer.draw_scene(
    surface,
    self.helicopter,
    self.obstacles
)
```

To this:

```
renderer.draw_scene(
```

```
        surface,  
        self.helicopter,  
        self.obstacles,  
        self.shield_active  
    )
```

That's the **only change needed in this step**.

The flow is now:

```
GameEngine  
  |  
  | self.shield_active  
  ↓  
renderer.draw_scene(...)  
  |  
  |— False → normal helicopter  
  |  
  |— True  → shield + helicopter
```

Step 6: Make sure the shield can actually be activated

Same file

```
game/game_engine.py
```

At the very top

You currently have:

```
import random
```

Add:

```
import pygame
```

So:

```
import random
import pygame
```

In `__init__()`

Find:

```
self.game_over = False
self.distance = 0
```

Add:

```
self.shield_active = False
```

So:

```
self.game_over = False
self.distance = 0
self.shield_active = False
```

Find `handle_keydown()`

Currently it's:

```
def handle_keydown(self, key):
    pass
```

Replace the **entire function** with:

```
def handle_keydown(self, key):
    if key == pygame.K_SPACE and not self.game_over:
        self.shield_active = True
```

Now:

SPACE

↓

shield_active = True

↓

renderer receives True

↓

shield appears

And remember, our collision code from Step 3 already does this when a collision occurs:

```
if self.shield_active:
    self.shield_active = False
```

So the shield disappears after absorbing the hit.

At this point, test ONLY the shield

Run:

```
python main.py
```

Then:

1. Press **Space** → shield should appear.
2. Fly into an obstacle → **no Game Over**.
3. Shield should disappear.
4. Hit another obstacle → **Game Over**.

Don't add the restart code yet. Once you confirm the shield works, we'll do the restart, and I'll give you the same exact **file** → **location** → **old code** → **new code** format.

YOU

this m y game_engine

"""

GameEngine: owns the helicopter and all obstacles.

Starter version: the helicopter moves and obstacles scroll by, but there's no collision detection at all yet (the helicopter can fly straight through obstacles harmlessly), no scoring, and no shield.

That's Tasks 2, 3, and 4. Movement also has known bugs (see game/helicopter.py) that Task 1 asks you to fix.

"""

```
import random
```

```
import pygame
```

```
from game.helicopter import Helicopter
```

```
from game.obstacle import Obstacle
```

```
from game.renderer import WIDTH, HEIGHT
```

```
SPAWN_INTERVAL_FRAMES = 90
```

```
GAP_HEIGHT = 150
```

```
WALL_WIDTH = 60
```

```
SCROLL_SPEED = 3
```

```
class GameEngine:
```

```
    def __init__(self):
```

```
        self.helicopter = Helicopter(x=100, y=HEIGHT / 2)
```

```
        self.obstacles = []
```

```
        self.frames_until_spawn = 0
```

```
        self.game_over = False
```

```
        self.distance = 0
```

```
        self.shield_active = False
```

```
    def _spawn_obstacle(self):
```

```
        margin = 60
```

```
        gap_y = random.randint(
```

```
            margin + GAP_HEIGHT // 2,
```

```
            HEIGHT - margin - GAP_HEIGHT // 2
```

```
        )
```

```
        self.obstacles.append(
```

```
Obstacle(  
    x=WIDTH,  
    gap_y=gap_y,  
    gap_height=GAP_HEIGHT,  
    wall_width=WALL_WIDTH,  
    screen_height=HEIGHT,  
    speed=SCROLL_SPEED  
)  
  
def handle_input(self, keys_pressed):  
    self.helicopter.handle_input(keys_pressed)  
  
def handle_keydown(self, key):  
    if key == pygame.K_r and self.game_over:  
        self.__init__()  
    return  
  
if key == pygame.K_SPACE and not self.game_over:  
    self.shield_active = True  
  
def update(self):  
    if self.game_over:  
        return  
  
    self.distance += SCROLL_SPEED  
  
    self.helicopter.update(HEIGHT)  
  
    self.frames_until_spawn -= 1  
  
    if self.frames_until_spawn <= 0:  
        self._spawn_obstacle()  
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES  
  
    for obstacle in self.obstacles:  
        obstacle.update()  
  
    heli_rect = self.helicopter.get_rect()  
  
    for obstacle in self.obstacles:  
        if (  
            heli_rect.colliderect(obstacle.get_top_rect())  
            or heli_rect.colliderect(obstacle.get_bottom_rect())
```

```
    ):  
    if self.shield_active:  
        self.shield_active = False  
    else:  
        self.game_over = True  
  
    break  
  
    self.obstacles = [  
        o for o in self.obstacles  
        if not o.is_off_screen()  
    ]
```

CHATGPT

Yep, I can see your current `game_engine.py` , and **you've already added the restart code too**. So let's not duplicate anything.

Your `game_engine.py` is currently in good shape for the shield logic:

Already correctly done

1. Shield state in `__init__` :

```
self.shield_active = False
```

2. Space activates the shield:

```
if key == pygame.K_SPACE and not self.game_over:
    self.shield_active = True
```

3. Collision consumes the shield:

```
if self.shield_active:
    self.shield_active = False
else:
    self.game_over = True
```

4. Restart is already implemented:

```
if key == pygame.K_r and self.game_over:
    self.__init__()
    return
```

That's actually a nice simple approach because `__init__()` resets:

```
helicopter
obstacles
game_over
distance
shield_active
```

```
spawn timer
```

So **do not change those parts right now.**

What you still need to change

There are only **two things left for the shield to actually work visually.**

Change 1: `renderer.py`

Open:

```
game/renderer.py
```

Find:

```
def draw_scene(surface, helicopter, obstacles):
```

Change it to:

```
def draw_scene(surface, helicopter, obstacles, shield_active=False):
```

Then find the existing helicopter drawing line:

```
pygame.draw.rect(surface, COLOR_HELI, helicopter.get_rect(), border_radius=4)
```

Put this **immediately before it:**

```
if shield_active:
    rect = helicopter.get_rect()
    pygame.draw.circle(
        surface,
        (30, 120, 255),
        rect.center,
        max(rect.width, rect.height)
    )
```

So the complete function should be:

```
def draw_scene(surface, helicopter, obstacles, shield_active=False):
    surface.fill(COLOR_BG)

    for obstacle in obstacles:
        pygame.draw.rect(surface, COLOR_OBSACLE, obstacle.get_top_rect())
        pygame.draw.rect(surface, COLOR_OBSACLE, obstacle.get_bottom_rect())

    if shield_active:
        rect = helicopter.get_rect()
        pygame.draw.circle(
            surface,
            (30, 120, 255),
            rect.center,
            max(rect.width, rect.height)
        )

    pygame.draw.rect(
        surface,
        COLOR_HELI,
        helicopter.get_rect(),
        border_radius=4
    )
```

Change 2: game_engine.py → draw()

Your game_engine.py that you pasted **doesn't include the** draw() **function** at the bottom.

You need to have this after your update() function:

```
def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.helicopter,
```

```
        self.obstacles,
        self.shield_active
    )

    renderer.draw_text(
        surface,
        font,
        f"Distance: {self.distance}",
        (10, 10)
    )

    if self.shield_active:
        renderer.draw_text(
            surface,
            font,
            "SHIELD ACTIVE",
            (10, 40)
        )

    if self.game_over:
        renderer.draw_banner(
            surface,
            font,
            f"GAME OVER - Distance: {self.distance}"
        )

        renderer.draw_text(
            surface,
            font,
            "Press R to Restart",
            (surface.get_width() // 2 - 100,
             surface.get_height() // 2 + 35)
        )
```

One important thing

Your pasted `game_engine.py` currently ends here:

```
self.obstacles = [  
    o for o in self.obstacles  
    if not o.is_off_screen()  
]
```

So **make sure the** `draw()` **function is still below that**. If you accidentally deleted it while making the previous changes, add the function above.

Then test

Run:

```
python main.py
```

Test in this order:

1. **Space** → shield appears.
2. Fly into an obstacle → shield disappears, game continues.
3. Hit another obstacle → `GAME OVER` .
4. Press **R** → fresh game.
5. Check distance → starts back at `0` .
6. Press Space again → new shield works.

If those work, **the coding requirements are basically complete**. Then we'll do one final pass through all five files against the README and catch anything we've missed before you make the before/after videos.
